

Prices come from a variety of sources.

All are reported in 2024 dollars using the Producer Price Index (PPI) or the Consumer Price Index (CPI), depending on where they are located along the supply chain.

For farmgate prices, we used the St. Louis Fed, Producer Price Index by Commodity for [soybeans](#) and [dry beans](#).

## Differentiated

We have limited data on prices for differentiated beans. Rather than providing a time series of prices, we provide an average and standard deviation of prices. All data is based on interviews and pricing documents.

## Blank table

Create a blank table to fill.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(openxlsx)
library(readxl)

# Define data frame
price <- bind_rows(
  expand_grid(
    type = "differentiated",
    organic = c("conventional", "organic"),
    supply_chain_location = c("farmgate", "aggregator/cleaner"),
    form = "dried"
  ),
```

```

expand_grid(
  type = "differentiated",
  organic = c("conventional", "organic"),
  supply_chain_location = c("primary_processor", "secondary_processor",
                           "wholesaler/distributor"),
  form = c("dried", "canned", "frozen")
)
) %>%
mutate(
  mean_price_lb = NA_real_,
  sd_price_lb = NA_real_,
  mean_price_kg = NA_real_,
  sd_price_kg = NA_real_
)

```

## Wholesale/distributor

We reviewed the product lists of seven distributors who were identified as carriers of local and regional products who also carry dried or canned beans, and who distribute into New York City and the Hudson Valley. These are reported in

```

library(knitr)
library(kableExtra)

```

Attaching package: 'kableExtra'

The following object is masked from 'package:dplyr':

group\_rows

```

## Table that looks nice
beans <- data.frame(
  Source = c("conventional", "conventional", "organic", "organic", "conventional", "organic",
            "conventional", "organic", "organic", "organic"),
  Supplier = c("1", "1",
              "2",
              "3",
              "2", "2",
              "4", "5", "5", "5"),
  Item = c("Canned beans, Black", "Canned beans, Kidney", "Dry Beans - Black",

```

```

      "Dry Beans - Pinto", "Dry Beans - Light Red Kidney", "Dry Beans - Light Red Kidney",
      "Dry Beans - Cranberry", "Dry Beans - Black", "Dry Beans - Light Red Kidney",
      "Dry Beans - Pinto"),
Unit = c("12 × 15.5 oz", "12 × 15.5 oz", "15 × 1 lb", "15 × 1 lb", "15 × 1 lb",
      "15 × 1 lb", "12 × 2 lb", "25 lb", "25 lb", "25 lb"),
Price_lb = c("$1.67", "$1.61", "$2.96", "$2.80", "$2.69", "$3.33",
      "$1.88", "$1.75", "$2.16", "$2.02"),
Notes = c("Range $1.58-$1.75", "Range $1.58-$1.63", "", "", "", "",
      "", "Average 3 sources", "Average 3 sources", "Average 3 sources")
)

kbl(
  beans,
  col.names = c("Source", "Supplier", "Item", "Unit", "$/lb", "Notes"),
  align      = c("c", "l", "l", "c", "r", "l"),
  caption    = "Distributor Price Ranges"
) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = FALSE,
    position          = "left",
    font_size         = 13
  ) |>
  column_spec(1, bold = TRUE, width = "1.5cm") |>
  column_spec(5, monospace = TRUE) |>
  row_spec(0, bold = TRUE, background = "#4a7c59", color = "white")

```

```

rm(beans)

## Make into a tibble
diff_wholesaler <- tibble(
  organic   = c("conventional", "conventional", "organic", "organic",
    "conventional", "organic", "conventional", "organic",
    "organic", "organic"),
  supplier  = c("Headwater Origins", "Headwater Origins", "Genesee Valley Bean Co.",
    "Seneca Grain & Bean", "Genesee Valley Bean Co.", "Genesee Valley Bean Co.",
    "Headwater Origins", "Multiple sources", "Multiple sources", "Multiple sources"),
  item      = c("Canned beans, Black", "Canned beans, Kidney", "Dry Beans - Black",
    "Dry Beans - Pinto", "Dry Beans - Light Red Kidney", "Dry Beans - Light Red Kidney",
    "Dry Beans - Cranberry", "Dry Beans - Black", "Dry Beans - Light Red Kidney",
    "Dry Beans - Pinto"),
  unit      = c("12 x 15.5 oz", "12 x 15.5 oz", "15 x 1 lb", "15 x 1 lb", "15 x 1 lb",

```

Table 1: Distributor Price Ranges

| Source                 | Supplier | Item                         | Unit         | \$/lb  | Notes               |
|------------------------|----------|------------------------------|--------------|--------|---------------------|
| con-<br>ven-<br>tional | 1        | Canned beans, Black          | 12 × 15.5 oz | \$1.67 | Range \$1.58–\$1.76 |
| con-<br>ven-<br>tional | 1        | Canned beans, Kidney         | 12 × 15.5 oz | \$1.61 | Range \$1.58–\$1.64 |
| or-<br>ganic           | 2        | Dry Beans – Black            | 15 × 1 lb    | \$2.96 |                     |
| or-<br>ganic           | 3        | Dry Beans – Pinto            | 15 × 1 lb    | \$2.80 |                     |
| con-<br>ven-<br>tional | 2        | Dry Beans – Light Red Kidney | 15 × 1 lb    | \$2.69 |                     |
| or-<br>ganic           | 2        | Dry Beans – Light Red Kidney | 15 × 1 lb    | \$3.33 |                     |
| con-<br>ven-<br>tional | 4        | Dry Beans – Cranberry        | 12 × 2 lb    | \$1.88 |                     |
| or-<br>ganic           | 5        | Dry Beans – Black            | 25 lb        | \$1.75 | Average 3 sources   |
| or-<br>ganic           | 5        | Dry Beans – Light Red Kidney | 25 lb        | \$2.16 | Average 3 sources   |
| or-<br>ganic           | 5        | Dry Beans – Pinto            | 25 lb        | \$2.02 | Average 3 sources   |

```

      "15 x 1 lb", "12 x 2 lb", "25 lb", "25 lb", "25 lb"),
price_lb = c(1.67, 1.61, 2.96, 2.80, 2.69, 3.33, 1.88, 1.75, 2.16, 2.02),
notes     = c("Range $1.58-$1.75", "Range $1.58-$1.63", NA, NA, NA, NA,
              NA, "Average 3 sources", "Average 3 sources", "Average 3 sources")
)

## Add type
diff_wholesaler <- diff_wholesaler %>%
  mutate(
    form = case_when(
      str_detect(item, "Canned") ~ "canned",
      TRUE ~ "dried"
    )
  )

## summarise
sum <- diff_wholesaler %>%
  group_by(organic, form) %>%
  summarise(
    mean = mean(price_lb),
    sd = sd(price_lb)
  )

```

`summarise()` has regrouped the output.

- i Summaries were computed grouped by organic and form.
- i Output is grouped by organic.
- i Use `summarise(.groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(organic, form))` for per-operation grouping (`?dplyr::dplyr_by`) instead.

```

rm(diff_wholesaler)

# Add wholesale data
price <- price %>%
  rows_update(
    sum %>%
      mutate(
        organic = map_chr(str_to_lower(organic),
                          ~unique(price$organic)[str_starts(unique(price$organic), .x)]),
        type = "differentiated",
        supply_chain_location = "wholesaler/distributor",
        mean_price_lb = mean,

```

```

    sd_price_lb = sd,
    mean_price_kg = mean / 0.453592,
    sd_price_kg = sd / 0.453592
  ) %>%
  select(type, organic, supply_chain_location, form,
         mean_price_lb, sd_price_lb, mean_price_kg, sd_price_kg),
  by = c("type", "organic", "supply_chain_location", "form")
)

## CV from wholesale data
cv_wholesale <- sum %>%
  mutate(cv = sd / mean) %>%
  summarise(cv = mean(cv)) %>%
  pull(cv)

```

## Secondary processor

Data on secondary processors comes from pricing sheets and interviews. Data is based on frozen black bean patties.

Organic: 5 oz patty, 24 patties/case, \$51 per case. Conventional: 3.30/lb

We don't have variation, so we rely on the variation we saw for wholesalers by taking  $cv = sd / \text{mean}$  and using that ratio to estimate a standard deviation.

```

# Organic mean/se
price <- price %>%
  mutate(
    mean_price_lb = case_when(
      organic == "organic" & supply_chain_location == "secondary_processor" & form == "frozen"
      TRUE ~ mean_price_lb),
    sd_price_lb = case_when(
      organic == "organic" & supply_chain_location == "secondary_processor" & form == "frozen"
      TRUE ~ sd_price_lb),
    mean_price_kg = case_when(
      organic == "organic" & supply_chain_location == "secondary_processor" & form == "frozen"
      TRUE ~ mean_price_kg),
    sd_price_kg = case_when(
      organic == "organic" & supply_chain_location == "secondary_processor" & form == "frozen"
      TRUE ~ sd_price_kg)
  )

# Conventional mean/se

```

```

price <- price %>%
  mutate(
    mean_price_lb = case_when(
      organic == "conventional" & supply_chain_location == "secondary_processor" & form == "dried" ~ mean_price_lb,
      TRUE ~ mean_price_lb),
    sd_price_lb = case_when(
      organic == "conventional" & supply_chain_location == "secondary_processor" & form == "dried" ~ sd_price_lb,
      TRUE ~ sd_price_lb),
    mean_price_kg = case_when(
      organic == "conventional" & supply_chain_location == "secondary_processor" & form == "dried" ~ mean_price_kg,
      TRUE ~ mean_price_kg),
    sd_price_kg = case_when(
      organic == "conventional" & supply_chain_location == "secondary_processor" & form == "dried" ~ sd_price_kg,
      TRUE ~ sd_price_kg)
  )

```

### Farmer & Aggregator/cleaner

We use data from the interviews to understand differentiated prices.

Breakdown on a 50lb bag: Grower receives \$52; Cleaner \$5; \$57 price out the door. We do not have variation, so we assume a similar variation as is seen in the wholesale data for differentiated beans.

All data was provided in 2023, we convert all prices to be in 2024 dollars, using the

```

## Add data
price <- price %>%
  rows_update(
    tibble(
      type = "differentiated",
      organic = c("conventional", "organic", "conventional", "organic"),
      supply_chain_location = c("farmgate", "farmgate",
                                "aggregator/cleaner", "aggregator/cleaner"),
      form = "dried",
      mean_price_lb = c(52/50, 52/50 * 2, 57/50, 57/50 * 2),
      sd_price_lb = mean_price_lb * cv_wholesale,
      mean_price_kg = mean_price_lb / 0.453592,
      sd_price_kg = sd_price_lb / 0.453592
    ),
    by = c("type", "organic", "supply_chain_location", "form")
  )

```

```
rm(sum)

# Rename diff price
diff_price <- price
```

## Commodity

Here we focus on price data for the last 10 years, where available. All prices are reported in 2024 dollars.

## Farmgate

For soybeans, we use the [global price of soybeans from the Federal Reserve Bank of St. Louis](#) (“PSOYBUSDM.csv”) for monthly price data. Units are reported as U.S. dollars per metric ton and are not seasonally adjusted. Value represents the benchmark prices which are representative of the global market. They are determined by the largest exporter of a given commodity. Prices are period averages in nominal U.S. dollars. Citation: International Monetary Fund, Global price of Soybeans [PSOYBUSDM], retrieved from FRED, Federal Reserve Bank of St. Louis; <https://fred.stlouisfed.org/series/PSOYBUSDM>, 2026.

## PPI

We use the [producer price index by commodity: Farm products: Soybeans](#) (“WPU01830131.csv”) to put all soybean price data in 2024 dollars. Citation: U.S. Bureau of Labor Statistics, Producer Price Index by Commodity: Farm Products: Soybeans [WPU01830131], retrieved from FRED, Federal Reserve Bank of St. Louis; <https://fred.stlouisfed.org/series/WPU01830131>, 2026.

We use the [producer price index by commodity: Farm products: drybeans](#) (“WPU0113011.csv”) to put all dry bean price data in 2024 dollars.

We compute an average yearly PPI and then use that to convert all dollars into 2024 dollars. We keep data starting in 2014 for soybeans. For dry bean data, we do not have information prior to 2015.

```
# create workbook
wb <- createWorkbook()

# Soy
soy_ppi <- read_csv("../data_raw/WPU01830131.csv") %>%
  filter(observation_date>= as.Date("2014-01-01"))
```

```
Rows: 936 Columns: 2
-- Column specification -----
Delimiter: ","
dbl  (1): WPU01830131
date (1): observation_date
```

i Use `spec()` to retrieve the full column specification for this data.  
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# PPI get average per year
soy_ppi <- soy_ppi %>%
  mutate(year = year(observation_date)) %>%
  group_by(year) %>%
  summarise(ppi = mean(WPU01830131, na.rm = TRUE)) %>%
  mutate(ppi_index_2024 = ppi[year == 2024] / ppi) %>%
  select(-ppi)

# Drybeans
drybeans_ppi <- read_csv("../data_raw/WPU0113011.csv")
```

```
Rows: 128 Columns: 2
-- Column specification -----
Delimiter: ","
dbl  (1): WPU0113011
date (1): observation_date
```

i Use `spec()` to retrieve the full column specification for this data.  
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# PPI get average per year
drybeans_ppi <- drybeans_ppi %>%
  mutate(year = year(observation_date)) %>%
  group_by(year) %>%
  summarise(ppi = mean(WPU0113011, na.rm = TRUE)) %>%
  mutate(ppi_index_2024 = ppi[year == 2024] / ppi) %>%
  select(-ppi)
```

## Soybeans

```
# import data
soy_price <- read_csv("../data_raw/PSOYBUSDM.csv") %>%
  filter(observation_date >= as.Date("2014-01-01"))
```

Rows: 300 Columns: 2

```
-- Column specification -----
Delimiter: ","
dbl  (1): PSOYBUSDM
date  (1): observation_date
```

i Use `spec()` to retrieve the full column specification for this data.  
 i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Rename soy price and add year
soy_price <- soy_price %>%
  rename(nominal_soy_price = PSOYBUSDM) %>%
  mutate(year = year(observation_date)) %>%
  select(year, everything())

# Join with ppi
soy_price <- soy_price %>%
  left_join(soy_ppi)
```

Joining with `by = join\_by(year)`

```
# Create real price and price per kg
soy_price <- soy_price %>%
  mutate(
    real_soy_price_metricton2024dollars = nominal_soy_price*ppi_index_2024,
    real_soy_price_perkg2024dollars = (nominal_soy_price*ppi_index_2024) / 1000
  ) %>%
  select(-ppi_index_2024)

# Rename date
soy_price <- soy_price %>%
  rename(date = observation_date)

# Add worksheet
addWorksheet(wb, "farmgate_soybean_price")
writeData(wb, "farmgate_soybean_price", soy_price)
```

## Drybeans

We do not have the same price series for the global price of drybeans (as we did for soybeans).

We utilize the [USDA AMS Bean Market News Summary \(Annual\)](#) from 2024, 2019, and 2018. All data are reported as pdfs, not in spreadsheet format.

We convert to 2024 prices using the [producer price index by commodity: Farm products: drybeans](#)("WPU0113011.csv").

The first full year of data available is 2015.

Data are reported for some of the larger bean producing states/groups of states. New York is not one that is reported. We rely on Michigan as they are often reported as having a similar growing climate for drybeans.

We use Grower Monthly Average Price Per Cwt By Crop Year Fob, Michigan. I copy and paste the tables of interest into ClaudeAI and copies the code that converts the table in a data frame.

```
# Copy and paste price data from PDFs via Claude
price_2024 <- tribble(
  ~Region,          ~Year,          ~SEP, ~OCT, ~NOV, ~DEC, ~JAN, ~FEB, ~MAR, ~APR,
  # BLACK - Michigan
  "BLACK - Michigan", "2020-2021", 28.00, 27.00, 27.50, 27.50, 32.88, 33.25, 34.00, 35.00,
  "BLACK - Michigan", "2021-2022", 46.00, 47.00, 47.75, 48.00, 48.50, 48.31, 48.25, 48.25,
  "BLACK - Michigan", "2022-2023", 47.06, 43.83, 42.82, 41.00, 43.00, 44.50, 45.00, 45.50,
  "BLACK - Michigan", "2023-2024", 41.67, 41.00, 41.00, 45.00, 45.00, 45.56, 40.00, 40.00,
  "BLACK - Michigan", "2024-2025", 33.50, 32.00, 35.00, 38.50, NA, NA, NA, NA,
)

price_2019 <- tribble(
  ~Region,          ~Year,          ~SEP, ~OCT, ~NOV, ~DEC, ~JAN, ~FEB, ~MAR, ~APR,
  # BLACK - Michigan
  "BLACK - Michigan", "2015-2016", 25.50, 23.25, 23.33, 23.50, 23.50, 23.63, 24.00, 24.63,
  "BLACK - Michigan", "2016-2017", 34.83, 35.00, 35.56, 35.75, NA, NA, NA, NA,
  "BLACK - Michigan", "2017-2018", 32.50, 30.00, 27.00, NA, NA, 27.00, NA, 28.50,
  "BLACK - Michigan", "2018-2019", 29.69, 30.10, 29.50, 29.50, 29.50, 29.50, 29.50, 29.25,
  "BLACK - Michigan", "2019-2020", 27.00, 27.50, 29.00, 29.00, NA, NA, NA, NA,
```

```

price_2018 <- tribble(
  ~Region,          ~Year,          ~SEP, ~OCT, ~NOV, ~DEC, ~JAN, ~FEB, ~MAR, ~APR,
  # BLACK - Michigan
  "BLACK - Michigan", "2014-2015", 32.00, 32.00, 32.00, 32.00, 31.25, 37.50, 36.60, 38.50,
  "BLACK - Michigan", "2015-2016", 25.50, 23.25, 23.33, 23.50, 23.50, 23.63, 24.00, 24.63,
  "BLACK - Michigan", "2016-2017", 34.83, 35.00, 35.56, 35.75, NA, NA, NA, NA,
  "BLACK - Michigan", "2017-2018", 32.50, 30.00, 27.00, NA, NA, 27.00, NA, 28.50,
  "BLACK - Michigan", "2018-2019", 29.69, 30.10, 29.50, 29.50, NA, NA, NA, NA,
)

# Combine and remove individual data frames
drybean_price <- bind_rows(price_2018, price_2019, price_2024) %>%
  mutate(n_missing = rowSums(is.na(across(SEP:AUG)))) %>%
  group_by(Region, Year) %>%
  slice_min(n_missing, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  select(-n_missing)

rm(price_2018, price_2019, price_2024)

# Reorganize to monthly data with calendar dates
drybean_price <- drybean_price %>%
  pivot_longer(SEP:AUG, names_to = "month", values_to = "price_cwt") %>%
  mutate(
    year_start = as.integer(str_sub(Year, 1, 4)),
    year_end   = as.integer(str_sub(Year, 6, 9)),
    year = if_else(month %in% c("SEP", "OCT", "NOV", "DEC"),
                  year_start, year_end),
    date = as.Date(paste(year, month, "01"), format = "%Y %b %d")
  ) %>%
  select(Region, date, price_cwt) %>%
  arrange(Region, date)

# Drop 2025 data
drybean_price <- drybean_price %>%
  filter(year(date) < 2025)

# Add ppi
drybean_price <- drybean_price %>%
  mutate(year = year(date)) %>%

```

```
left_join(drybeans_ppi)
```

Joining with `by = join\_by(year)`

```
# We do not have a PPI for 2014 (series starts in 2015), drop 2015 data
drybean_price <- drybean_price %>%
  filter(year!=2014) %>%
  mutate(
    real_price_cwt_2024dollars = price_cwt * ppi_index_2024,
    real_price_kg_2024dollars = real_price_cwt_2024dollars / 45.359)

# Drop data we don't need
drybean_price <- drybean_price %>%
  select(!c(Region, ppi_index_2024)) %>%
  select(date, year, everything())

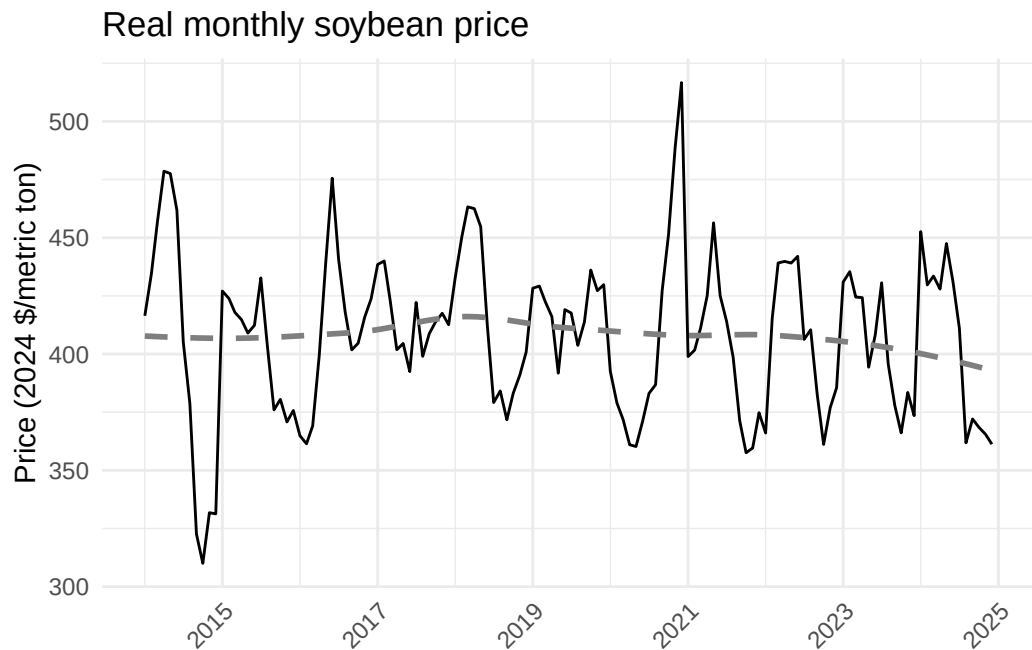
# Rename
drybean_price <- drybean_price %>%
  mutate(
    nominal_drybean_price = price_cwt
  )

# Add worksheet
addWorksheet(wb, "farmgate_drybean_price")
writeData(wb, "farmgate_drybean_price", drybean_price)
```

## Figures

```
# Soybeans
ggplot(soy_price, aes(x = date,
                      y = real_soy_price_metricton2024dollars)) +
  geom_line() +
  geom_smooth(method = "loess", se = FALSE, linetype = "dashed", color = "grey50") +
  labs(
    x = NULL,
    y = "Price (2024 $/metric ton)",
    title = "Real monthly soybean price"
  ) +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y",
               limits = as.Date(c("2014-01-01", "2024-12-31")))) +
```

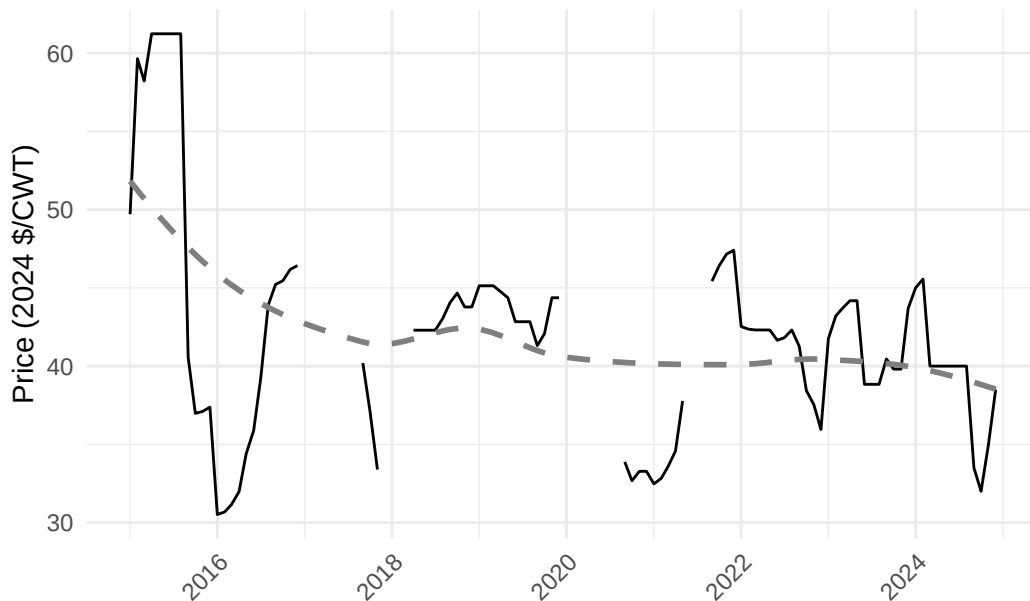
```
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# Drybeans
ggplot(drybean_price, aes(x = date, y = real_price_cwt_2024dollars)) +
  geom_line() +
  geom_smooth(method = "loess", se = FALSE, linetype = "dashed", color = "grey50") +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
  labs(
    x = NULL,
    y = "Price (2024 $/CWT)",
    title = "Real monthly dry bean price over time"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Warning: Removed 22 rows containing non-finite outside the scale range (`stat\_smooth()`).

Real monthly dry bean price over time



## Organic

We use data from [The Risk Management Agency \(RMA\)](#) on approved the 2023 CY projected prices shown below for the following plans of insurance: Yield Protection, Revenue Protection, and Revenue Protection with Harvest Price Exclusion.

The projected price per pound for drybeans is double for organic compared to conventional. We double all conventional prices.

For soybeans, we use [Price discovery data from the RMA](#).

```
# import soy data
soy_organic <- read_xlsx("../data_raw/RMA_soybeans.xlsx",
                        sheet = "Current Periods") %>%
  janitor::clean_names()

# Select data of interest
soy_organic <- soy_organic %>%
  select(commodity_year, practice_name, state_name, projected_price)

# compare prices for organic and conventional for the same state year
soy_organic <- soy_organic %>%
  group_by(commodity_year, practice_name) %>%
  summarise(
```

```

    price = mean(projected_price)
  ) %>%
  mutate(
    organic_conventional =
      price[practice_name=="Organic"] /
      price[practice_name=="Conventional"]
  ) %>%
  slice(1) %>%
  pull(organic_conventional)

```

`summarise()` has regrouped the output.  
 i Summaries were computed grouped by commodity\_year and practice\_name.  
 i Output is grouped by commodity\_year.  
 i Use `summarise(.groups = "drop\_last")` to silence this message.  
 i Use `summarise(.by = c(commodity\_year, practice\_name))` for per-operation grouping (`?dplyr::dplyr\_by`) instead.

```

# Get price/per kg. for drybeans and add organic (*2)
drybean_price <- drybean_price %>%
  mutate(across(starts_with("real"),
    ~.*2,
    .names = "organic_{.col}")
  )

# Get price/per kg. for soybeans
soy_price <- soy_price %>%
  mutate(across(starts_with("real"),
    ~.*2,
    .names = "organic_{.col}")
  )

# Join
farmgate_price <- left_join(drybean_price, soy_price,
  by = c("date", "year"))

```

## Retail (consumer price)

Average Price: Beans, Dried, Any Type, All Sizes (Cost per Pound/453.6 Grams) in U.S. City Average("APU0000714233.csv"). Includes all dried organic and non-organic beans, regardless of package size.

Average consumer prices are calculated for household fuel, motor fuel, and food items from prices collected for the Consumer Price Index (CPI). Average prices are best used to measure the price level in a particular month, not to measure price change over time. It is more appropriate to use CPI index values for the particular item categories to measure price change. Prices, except for electricity, are collected monthly by BLS representatives in the 75 urban areas priced for the CPI.

We report all prices in 2024 dollars using the Federal Reserve of St. Louis [Consumer Price Index for All Urban Consumers: All Items in U.S. City Average](#) ("CPIAUCSL.csv")

## CPI

```
# Import data
cpi <- read_csv("../data_raw/CPIAUCSL.csv")
```

Rows: 952 Columns: 2

```
-- Column specification -----
Delimiter: ","
dbl  (1): CPIAUCSL
date  (1): observation_date
```

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Keep last 10 years and create index to convert to 2024 dollars
cpi <- cpi %>%
  filter(observation_date >= as.Date("2014-01-01"),
         observation_date <= as.Date("2024-12-31")) %>%
  mutate(year = year(observation_date)) %>%
  group_by(year) %>%
  summarise(cpi = mean(CPIAUCSL, na.rm = TRUE)) %>%
  mutate(cpi_index_2024 = cpi[year == 2024] / cpi) %>%
  select(-cpi)
```

## Drybeans

```
# Import bean data
retail_beans <- read_csv("../data_raw/APU0000714233.csv")
```

```

Rows: 370 Columns: 2
-- Column specification -----
Delimiter: ","
dbl (1): APU0000714233
date (1): observation_date

```

i Use ``spec()`` to retrieve the full column specification for this data.  
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```

# Keep relevant years and rename
retail_beans <- retail_beans %>%
  rename(nominal_drybeans_cost_perlb = APU0000714233) %>%
  filter(year(observation_date) >= 2014,
         year(observation_date) <= 2024) %>%
  mutate(year = year(observation_date)) %>%
  left_join(cpi) %>%
  mutate(
    real_drybeans_cost_perlb_2024dollars = nominal_drybeans_cost_perlb * cpi_index_2024,
    real_drybeans_cost_perkg_2024dollars = real_drybeans_cost_perlb_2024dollars / 0.453592
  ) %>%
  select(year, everything(), -cpi_index_2024)

```

Joining with ``by = join_by(year)``

## Figures

```

ggplot(retail_beans, aes(x = observation_date,
                        y = real_drybeans_cost_perlb_2024dollars)) +
  geom_line() +
  geom_smooth(method = "loess", se = FALSE, linetype = "dashed", color = "grey50") +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y",
              limits = as.Date(c("2014-01-01", "2024-12-31"))) +
  labs(
    x = NULL,
    y = "Price (2024 $/lb)",
    title = "Real retail dry bean price over time"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

Warning: Removed 2 rows containing non-finite outside the scale range (``stat_smooth()``).

